```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
url = "https://raw.githubusercontent.com/laxmimerit/All-CSV-ML-Data-Files-Do

data = pd.read_csv(url)
data.head()
```

	2401	Borderlands	Positive	im getting on borderlands and i will murder you all ,
0	2401	Borderlands	Positive	I am coming to the borders and I will kill you...
1	2401	Borderlands	Positive	im getting on borderlands and i will kill you ...
2	2401	Borderlands	Positive	im coming on borderlands and i will murder you...
3	2401	Borderlands	Positive	im getting on borderlands 2 and i will murder ...
4	2401	Borderlands	Positive	im getting into borderlands and i can murder y...

```
# The original data was loaded without specifying header=None,
# causing the first row of data to be interpreted as column names.
# Based on the CSV structure, the columns are typically:
# Column 0: ID
# Column 1: Topic
# Column 2: Sentiment
# Column 3: Text

# Rename the columns to their intended names
data.columns = ['id', 'topic', 'sentiment', 'text']

# Now select the 'text' and 'sentiment' columns
data = data[['text', 'sentiment']]
data.head()
```

	text	sentiment
0	I am coming to the borders and I will kill you...	Positive
1	im getting on borderlands and i will kill you ...	Positive
2	im coming on borderlands and i will murder you...	Positive
3	im getting on borderlands 2 and i will murder ...	Positive
4	im getting into borderlands and i can murder y...	Positive

```
# Ensure the sentiment column is treated as string type before applying stri
data['sentiment'] = data['sentiment'].astype(str)
# Clean the sentiment column: convert to lowercase and strip whitespace for
data['sentiment'] = data['sentiment'].str.lower().str.strip().map({'positive
# Drop rows where sentiment could not be mapped (e.g., if there were other v
data.dropna(subset=['sentiment'], inplace=True)
# Convert sentiment to integer type
data['sentiment'] = data['sentiment'].astype(int)
```

```
X = data['text']
y = data['sentiment']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)

# Fill any NaN values in X_train and X_test with an empty string before vect
X_train_vec = vectorizer.fit_transform(X_train.fillna(''))
X_test_vec = vectorizer.transform(X_test.fillna(''))
```

```
data.isnull().sum()
```

	0
text	0
sentiment	0

dtype: int64

```
data = data.dropna()
```

```
data = data[['text', 'sentiment']]
```

```
data['sentiment'] = data['sentiment'].map({'positive': 1, 'negative': 0})
```

```
data = data.dropna()
```

```
X = data['text']
y = data['sentiment']
```

```
import pandas as pd

url = "https://raw.githubusercontent.com/laxmimerit/All-CSV-ML-Data-File

data = pd.read_csv(url)
```

```
data.head()
```

	2401	Borderlands	Positive	im getting on borderlands and i will murder you all ,
0	2401	Borderlands	Positive	I am coming to the borders and I will kill you...
1	2401	Borderlands	Positive	im getting on borderlands and i will kill you ...
2	2401	Borderlands	Positive	im coming on borderlands and i will murder you...
3	2401	Borderlands	Positive	im getting on borderlands 2 and i will murder ...
4	2401	Borderlands	Positive	im getting into borderlands and i can murder y...

```
print(data.columns)
```

```
Index(['2401', 'Borderlands', 'Positive',
       'im getting on borderlands and i will murder you all ,'],
      dtype='object')
```

```
data.columns = data.columns.str.lower()
```

```
print(data.head())
```

```
   2401  borderlands  positive  \
0  2401  Borderlands  Positive
1  2401  Borderlands  Positive
2  2401  Borderlands  Positive
3  2401  Borderlands  Positive
4  2401  Borderlands  Positive

  im getting on borderlands and i will murder you all ,
0  I am coming to the borders and I will kill you...
1  im getting on borderlands and i will kill you ...
2  im coming on borderlands and i will murder you...
3  im getting on borderlands 2 and i will murder ...
4  im getting into borderlands and i can murder y...
```

```
data.columns = data.columns.str.strip().str.lower()
print(data.columns)
```

```
Index(['2401', 'borderlands', 'positive',
       'im getting on borderlands and i will murder you all ,'],
      dtype='object')
```

```
import pandas as pd

url = "https://raw.githubusercontent.com/laxmimerit/All-CSV-ML-Data-Files-Do

data = pd.read_csv(url)
data.head()
```

	2401	Borderlands	Positive	im getting on borderlands and i will murder you all ,
0	2401	Borderlands	Positive	I am coming to the borders and I will kill you...
1	2401	Borderlands	Positive	im getting on borderlands and i will kill you ...
2	2401	Borderlands	Positive	im coming on borderlands and i will murder you...
3	2401	Borderlands	Positive	im getting on borderlands 2 and i will murder ...
4	2401	Borderlands	Positive	im getting into borderlands and i can murder y...

```
data.columns = ['id', 'topic', 'sentiment', 'text']
```

```
data = data[['text', 'sentiment']]
```

```
data['sentiment'] = data['sentiment'].str.lower().str.strip()
```

```
data = data[data['sentiment'].isin(['positive', 'negative'])]
```

```
data['sentiment'] = data['sentiment'].map({
    'positive': 1,
    'negative': 0
})
```

```
data = data.dropna()
print(data.shape)
```

```
(43555, 2)
```

```
from sklearn.model_selection import train_test_split

X = data['text']
y = data['sentiment']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

```
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression # Import LogisticRegress
```

```
# Initialize and fit the model
model = LogisticRegression(max_iter=1000) # Added max_iter for convergence,
model.fit(X_train_vec, y_train)

y_pred = model.predict(X_test_vec)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.868442199517851
```

```
def predict(text):
    vec = vectorizer.transform([text])
    result = model.predict(vec)

    if result[0] == 1:
        print(" Positive")
    else:
        print(" Negative")

predict("I love this game")
predict("This is very bad experience")
```

```
 Positive
 Negative
```